

ui.chip 全面详解（基于 NiceGUI 文档）

ui.chip 是 NiceGUI 中用于展示标签、分类或可交互选项的轻量级组件，基于 Quasar 的 QChip 组件实现。它支持点击、选择、移除等核心交互，可通过图标、颜色、样式定制适配多种场景，既适用于静态标签展示（如分类标签），也可实现动态交互（如可选择标签组、可删除标签列表），是界面信息分类与快速操作的常用组件。以下从核心特性、基础用法、高级功能等维度展开全面解析。

一、核心基础

1. 组件本质与核心特性

- 底层依赖：基于 Quasar 的 QChip 组件，继承其紧凑布局、圆角样式和丰富的交互能力。
- 核心功能：支持「点击触发」「选中切换」「一键移除」三种核心交互，同时支持图标、颜色、文本的灵活定制。
- 状态管理：移除的芯片并未删除，仅将 `value` 属性设为 `False`，支持通过 `set_value(True)` 恢复显示。

2. 初始化参数（核心配置）

参数名	类型	说明	默认值
text	str	芯片的文本内容（支持简短文本或标识）	""
icon	str / None	芯片左侧显示的图标名称（符合 Quasar 图标规范）	None
color	str / None	芯片背景色（支持 Quasar 颜色、Tailwind 颜色、CSS 颜色）	"primary"
text_color	str / None	文本颜色（优先级高于背景色默认文本色）	None
on_click	Callable	点击芯片时触发的回调函数（设置后芯片变为可点击状态）	-
selectable	bool	是否支持选中状态切换（选中后有视觉高亮）	False
selected	bool	初始是否为选中状态（仅 <code>selectable=True</code> 时有效）	False
on_selection_change	Callable	选中状态变化时触发的回调函数	-
removable	bool	是否支持移除（显示右侧“×”按钮，点击可隐藏芯片）	False
on_value_change	Callable	芯片移除 / 恢复时触发的回调函数（ <code>value</code> 属性变化时触发）	-

二、基础使用示例

1. 多种交互类型的芯片

展示可点击、可选择、可移除、自定义样式及禁用状态的芯片，覆盖核心基础用法：

```
from nicegui import ui

with ui.row().classes('gap-1'):
    # 可点击芯片（点击触发通知）
    ui.chip('Click me', icon='ads_click', on_click=lambda: ui.notify('Clicked'))
    # 可选择芯片（点击切换选中状态）
    ui.chip('Selectable', selectable=True, icon='bookmark', color='orange')
    # 可移除芯片（右侧显示“×”按钮）
    ui.chip('Removable', removable=True, icon='label', color='indigo-3')
    # 自定义样式芯片（轮廓+方形）
    ui.chip('Styled', icon='star', color='green').props('outline square')
    # 禁用芯片（不可交互，视觉灰度）
    ui.chip('Disabled', icon='block', color='red').set_enabled(False)

ui.run()
```

效果：五个芯片横向排列，分别对应不同交互类型，样式和功能各有区分，点击、选择、移除操作均有对应视觉反馈。

2. 动态标签列表（可添加 / 恢复芯片）

实现可动态添加新标签、移除现有标签，且支持恢复已移除标签的功能，适用于标签管理场景：

```
from nicegui import ui

def add_chip():
    # 向芯片容器中添加新芯片（可移除）
    with chips:
        ui.chip(label_input.value, icon='label', color='silver', removable=True)
    label_input.value = '' # 清空输入框

# 输入框：输入标签文本，按回车或点击加号按钮添加
label_input = ui.input('Add label').on('keydown.enter', add_chip)
with label_input.add_slot('append'):
    ui.button(icon='add', on_click=add_chip).props('round dense flat')

# 芯片容器（初始包含一个默认标签）
with ui.row().classes('gap-0') as chips:
    ui.chip('Label 1', icon='label', color='silver', removable=True)

# 恢复所有已移除的芯片（将 value 设为 True）
ui.button('Restore removed chips', icon='unarchive',
          on_click=lambda: [chip.set_value(True) for chip in chips]) \
    .props('flat')
```

```
ui.run()
```

效果：输入框输入文本后，按回车或点击加号可添加新标签；每个标签右侧有“×”按钮，点击可移除；点击“Restore removed chips”按钮可恢复所有已移除的标签。

三、核心功能与进阶用法

1. 选中状态监听与控制

通过 `selectable=True` 开启选中功能，结合 `on_selection_change` 监听状态变化，或通过 `set_selected()` 手动控制选中状态：

```
from nicegui import ui

# 可选择芯片，初始未选中
selectable_chip = ui.chip(
    'Toggle Selection',
    selectable=True,
    icon='check',
    color='blue',
    on_selection_change=lambda e: ui.notify(f'Selected: {e.value}')
)

# 手动控制选中/取消选中的按钮
with ui.row().classes('mt-4'):
    ui.button('Select', on_click=lambda: selectable_chip.set_selected(True))
    ui.button('Deselect', on_click=lambda: selectable_chip.set_selected(False))

ui.run()
```

效果：点击芯片可切换选中状态，状态变化时触发通知；点击下方按钮可手动控制芯片的选中 / 取消选中。

2. 动态修改芯片属性（文本、图标、颜色）

通过组件实例的方法，动态更新芯片的文本、图标、颜色等属性，适用于数据实时变化的场景：

```
from nicegui import ui

# 创建初始芯片
chip = ui.chip('Original', icon='edit', color='gray')

# 动态修改文本和颜色
def update_text_color():
    chip.set_text('Updated')
    chip.set_color('purple')

# 动态修改图标
def update_icon():
    chip.set_icon('refresh' if chip.icon == 'edit' else 'edit')

# 控制按钮
with ui.row().classes('mt-4 gap-2'):
```

```
ui.button('Update Text & Color', on_click=update_text_color)
ui.button('Toggle Icon', on_click=update_icon)

ui.run()
```

效果：点击“Update Text & Color”按钮，芯片文本变为“Updated”、颜色变为紫色；点击“Toggle Icon”按钮，芯片图标在“edit”和“refresh”之间切换。

3. 数据绑定（动态同步状态与属性）

通过 `bind_*` 系列方法，将芯片的选中状态、文本、颜色等与数据对象绑定，实现数据驱动界面更新：

```
from nicegui import ui

class TagState:
    def __init__(self):
        self.tag_text = 'Bind Demo'
        self.is_selected = False
        self.tag_color = 'green'

state = TagState()

# 绑定文本、选中状态和颜色
chip = ui.chip(
    text=state.tag_text,
    selectable=True,
    selected=state.is_selected,
    color=state.tag_color
).bind_text(state, 'tag_text') \
  .bind_selected(state, 'is_selected') \
  .bind_color(state, 'tag_color')

# 修改绑定数据的按钮
with ui.row().classes('mt-4 gap-2'):
    ui.button('Change Text', on_click=lambda: setattr(state, 'tag_text', 'New Bind Text'))
    ui.button('Toggle Selection', on_click=lambda: setattr(state, 'is_selected', not
state.is_selected))
    ui.button('Change Color', on_click=lambda: setattr(state, 'tag_color', 'orange'))

ui.run()
```

效果：点击按钮修改 `state` 对象的属性，芯片的文本、选中状态、颜色会自动同步更新，无需手动调用 `update()`。

4. 样式深度定制（Props 与 CSS 类）

通过 `props` 设置 Quasar 组件属性，结合 `classes` 应用 Tailwind/CSS 样式，实现个性化外观：

```
from nicegui import ui

with ui.row().classes('gap-3'):
    # 方形+轮廓+大尺寸芯片
    ui.chip('Square Outline', icon='cube', color='teal').props('square outline size=lg')
    # 圆角+渐变背景（通过 style 自定义 CSS）
    ui.chip('Gradient', icon='sparkles', text_color='white').style('background: linear-gradient(45deg, #667eea 0%, #764ba2 100%)')
    # 带悬停效果的芯片（Tailwind 类）
    ui.chip('Hover Effect', icon='hand-pointer', color='blue').classes('hover:scale-105 transition-transform')

ui.run()
```

效果：三个芯片分别展示方形轮廓、渐变背景、悬停缩放效果，样式灵活多样，适配不同设计需求。

四、核心属性与方法

1. 常用属性

属性名	类型	说明
classes	Classes[Self]	CSS 类（支持 Tailwind/Quasar 样式）
enabled	BindableProperty	是否启用（禁用后无法触发点击、选择、移除操作）
html_id	str	HTML 元素 ID（2.16.0+ 版本支持）
icon	BindableProperty	图标名称（可绑定数据动态修改）
selected	BindableProperty	选中状态（仅 <code>selectable=True</code> 时有效，可绑定数据）
text	BindableProperty	文本内容（可绑定数据动态修改）
value	BindableProperty	显示状态（True = 显示，False = 隐藏，移除芯片时自动设为 False）
visible	BindableProperty	是否可见（可绑定数据）
color	str	背景色（支持 Quasar/Tailwind/CSS 颜色）
text_color	str	文本颜色（支持 Quasar/Tailwind/CSS 颜色）

2. 关键方法

(1) 状态控制

- `disable()`：禁用芯片（不可交互，视觉灰度）
- `enable()`：启用芯片
- `set_enabled(value: bool)`：设置启用状态（True/False）
- `set_selected(selected: bool)`：设置选中状态（仅 `selectable=True` 时生效）
- `set_value(value: bool)`：设置显示状态（True = 显示，False = 隐藏，用于恢复移除的芯片）

- `set_visibility(visible: bool)`: 设置芯片可见性

(2) 属性修改

- `set_text(text: str)`: 动态修改文本内容
- `set_icon(icon: str | None)`: 动态修改图标
- `set_color(color: str)`: 动态修改背景色
- `update()`: 触发客户端界面更新（修改属性后需手动调用，绑定数据时无需）

(3) 事件与绑定

- `on_click(callback)`: 绑定点击事件（设置后芯片变为可点击状态）
- `on_selection_change(callback)`: 绑定选中状态变化事件（`e.value` 为当前选中状态）
- `on_value_change(callback)`: 绑定显示状态变化事件（`e.value` 为当前显示状态）
- `on(type: str, handler)`: 订阅任意 DOM 事件（如 `mouseover`、`mousedown`）
- `bind_selected(target_object, target_name)`: 双向绑定选中状态到目标对象属性
- `bind_text_from(target_object, target_name)`: 单向绑定文本从目标对象
- `bind_value(target_object, target_name)`: 双向绑定显示状态到目标对象属性

(4) 其他实用方法

- `tooltip(text: str)`: 为芯片添加悬停提示
- `delete()`: 彻底删除芯片及所有子元素（区别于 `set_value(False)` 的隐藏）
- `mark(*markers)`: 添加标记（用于测试或元素查询）
- `clear()`: 移除芯片内的所有子元素（如嵌套的图标、文本）

五、使用场景与注意事项

1. 适用场景

- 分类标签：如文章分类、商品标签、数据筛选条件等静态标签展示。
- 可交互选项：如多选项切换（可选择芯片组）、快速操作入口（可点击芯片）。
- 标签管理：如用户自定义标签列表（可添加 / 移除芯片）、已选条件标签（可删除单个条件）。

2. 关键注意事项

- 移除逻辑：`removable=True` 时，点击“×”按钮仅隐藏芯片（`value=False`），并未删除，可通过 `set_value(True)` 恢复。
- 选中状态限制：`selected` 属性仅在 `selectable=True` 时有效，未开启选择功能时，设置 `selected=True` 无视觉反馈。
- 颜色优先级：`text_color` 参数优先级高于背景色默认文本色（如深色背景默认白色文本，设置 `text_color='black'` 后强制显示黑色）。
- 版本兼容性：`html_id` 属性仅在 2.16.0+ 版本支持，`bind_*` 方法的 `strict` 参数在 3.0.0+ 版本支持，使用时需确认版本。

- 文本长度：芯片设计为紧凑组件，建议文本长度控制在 2-10 个字符，过长会导致样式变形或换行。

六、进阶示例：多选择标签组（带选中状态统计）

实现可选择的标签组，实时统计选中的标签数量，适用于筛选、分类选择等场景：

```
from nicegui import ui

class SelectionState:
    def __init__(self):
        self.selected_count = 0

state = SelectionState()

# 标签组容器
with ui.column().classes('gap-3'):
    ui.label(f'Selected: {state.selected_count}').bind_text(state, 'selected_count')

# 可选择标签组
with ui.row().classes('gap-2'):
    tags = ['Python', 'JavaScript', 'Java', 'C++', 'Go']
    for tag in tags:
        # 每个标签绑定选中状态变化事件，更新统计数
        ui.chip(
            tag,
            selectable=True,
            icon='code',
            color='indigo'
        ).on_selection_change(
            lambda e: setattr(
                state,
                'selected_count',
                state.selected_count + 1 if e.value else state.selected_count - 1
            )
        )

ui.run()
```

效果：标签组包含 5 个可选择标签，点击标签切换选中状态，上方标签实时显示当前选中的标签数量，选中状态变化时自动更新统计数。